Smart Budget Planner

Optimizing Grocery Shopping Using the 0/1 Knapsack Algorithm

MSML606 Extra Credit Project 2

Anoushka Anand & FNU Hardik

01

THE PROBLEM

Shopping on a Budget is Harder Than It Looks

You walk into Costco with $100. There are 50+ items on the shelves. Which combination gives you the best value without going over your budget?

x Manually comparing every possible combination is computationally infeasible

x Guessing leads to suboptimal choices — you spend more or get less value

\+ **The 0/1 Knapsack algorithm finds the provably optimal solution instantly**

DATASET

The Costco Grocery Dataset

Source: Kaggle scraped from Costco's online marketplace kaggle.com/datasets/elvinrustam/grocery-dataset

54

Grocery Items

across 7 categories

4.3 to 4.8

Customer Rating Range

used as the item value score

$1.99 to $40

Price Range

converted to cents for DP

Key Fields

Field	Description	Role in Algorithm
`title`	Product name	Shown to user in results
`price`	Price in USD	Knapsack weight (the cost constraint)
`rating`	1 to 5 star customer rating	Knapsack value (what we maximize)
`subcategory`	Grocery category	Used for UI category filter
`vegetarian`	True or False flag	Used for vegetarian mode filter

What Is the 0/1 Knapsack Problem?

In plain English: You have a budget (the knapsack capacity). Each grocery item has a price (weight) and a rating score (value). You want to choose items that fit within your budget while making the total rating as high as possible. Each item can only be bought once, it is either included or excluded (0 or 1).

INPUT	ALGORITHM	OUTPUT
Budget (e.g. $100)	Evaluates all item combinations	Best combination of items
List of grocery items	Finds globally optimal set	Highest total rating score
Each item's price and rating score	Guaranteed exact answer	Total cost within budget

Time complexity: O(n x W) where n = number of items, W = budget in cents

ALGORITHM PART 2 OF 3

How the DP Table Is Built

```
dp[i][w]  =  max( dp[i-1][w],   dp[i-1][w - price_i] + value_i )
```

For each item i and each budget w: either skip item i and keep the previous best, or include it and add its value to the best result without it.

Item Added	Budget (dollars)							
	$0	$2	$4	$6	$8	$10	$12	$15
(no items)	0	0	0	0	0	0	0	0
Bananas $2 / 45 pts	0	**45**	**45**	**45**	**45**	**45**	**45**	**45**
Eggs $8 / 48 pts	0	45	45	45	**48**	**93**	**93**	**93**
Spinach $6 / 46 pts	0	45	45	**46**	**91**	93	**94**	**139**
Rice $9 / 47 pts	0	45	45	46	91	93	94	139

= value improved when item was added

After filling the table, the algorithm backtracks from the bottom-right to recover which items were selected.

Implementation Details

Why Convert Prices to Cents?

Dynamic programming requires the budget to be a discrete integer. You cannot use a decimal like 12.99 as an array index.

Solution: multiply every price by 100 and round.

```
$12.99  -->  1299 cents
$6.49   -->   649 cents
$100.00 --> 10000 cents
```

This makes the table index exact and avoids all floating-point rounding errors.

How Backtracking Recovers the Items

Once the table is filled we know the maximum score but not yet which items produce it. Backtracking recovers the exact selection:

1. Start at dp[n][W], the bottom-right cell
2. If dp[i][w] differs from dp[i-1][w], item i was included
3. Subtract item i's price from w, move up one row
4. Repeat until row i = 0
5. The collected items form the optimal selection

APPLICATION

What the App Does

Budget Slider

Set any budget from $10 to $300. The algorithm re-runs instantly and updates the optimal item selection every time the slider moves.

Must-Have Items

Pin specific items that must always be included. Their cost is deducted first and the algorithm optimizes the remaining budget.

Vegetarian Mode

A single toggle removes all non-vegetarian items from the dataset before the algorithm runs, so the result respects dietary preferences.

Subcategory Filter

Filter by Dairy, Produce, Proteins, Grains, Snacks, Pantry, or Beverages. The knapsack runs only on the filtered subset.

EXAMPLE

Given a $100 Budget Algorithm Output

Item	Category	Price	Rating
Organic Whole Milk (1 gal)	Dairy	$6.99	4.7
Large Eggs (24-count)	Dairy	$8.49	4.8
Baby Spinach (2 lb)	Produce	$5.99	4.6
Blueberries (3 lb)	Produce	$9.99	4.8
Rotisserie Chicken	Proteins	$7.99	4.8
Olive Oil Extra Virgin (2L)	Pantry	$19.99	4.8
Peanut Butter (2 jars)	Proteins	$11.99	4.7
Jasmine Rice (25 lb)	Grains	$18.99	4.7
+ 7 more items			

SPENT

$99.85

AVG RATING

4.54

ITEMS SELECTED

15

LIMITATIONS & FUTURE SCOPE

What This Project Does Not Cover and Where It Could Go

Current Limitations

1. The dataset is a small, manually curated subset of 54 items. It is not a live feed from Costco, so prices and availability may not reflect the current store.
2. Customer rating is used as a proxy for value, but it does not capture personal preference, nutritional content, or necessity.
3. The algorithm assumes every item in the list is available and in stock, with no real-world inventory constraints.
4. The DP table grows with budget size in cents. A very large budget with many items would slow the algorithm noticeably.

Future Scope

1. Connect to a live grocery API so prices and availability update in real time.
2. Replace the fixed rating score with a personalized value function where users set their own weights across nutrition, price, and brand preference.
3. Add a nutritional optimization mode that maximizes protein or fiber content instead of customer rating.
4. Extend to a mobile app so users can run the planner in the store while shopping.

PROJECT INFO

Tech Stack

Tech Stack	
Language	**Python 3**
UI Framework	**Streamlit**
Algorithm	**0/1 Knapsack (custom DP)**
Data Handling	**Pandas + CSV**
Dataset	**Kaggle Costco Grocery**

Thank You

Smart Budget Planner

0/1 Knapsack Costco Grocery Dataset Streamlit Python 3

GitHub: https://github.com/aanand1811/knapsack-budget-planner

Dataset: https://www.kaggle.com/datasets/elvinrustam/grocery-dataset/data

Anoushka Anand & FNU Hardik